



Relatório Gerencial

Roteirização Preditiva & Digital Twin

Documento gerado em: 2026-04-28 09:52:39.676273



Relatório Gerencial de Roteirização Preditiva

Eduardo Lopes Jonker *Disciplina: Sistemas Inteligentes UDESC* Relatório gerado em: 2026-04-28 09:52:39.676273

Este documento consolida os resultados do sistema de roteirização preditiva, oferecendo uma prova de conceito visual e quantitativa da metodologia aplicada. O objetivo é fornecer uma visão conclusiva e menos abstrata sobre a otimização logística.

7. Detalhamento de Implementação e Fundamentação Matemática

Esta seção oferece uma visão mais aprofundada da implementação dos algoritmos e das equações matemáticas que regem o sistema.

7.1. Motor Preditivo (Prophet)

Equação Fundamental do Prophet

O Prophet utiliza um modelo de decomposição aditiva de séries temporais, representado pela equação:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

Onde:

- $y(t)$: O volume de impressões previsto no tempo t .
- $g(t)$: A componente de tendência, representando o crescimento ou queda não-periódica.
- $s(t)$: A componente de sazonalidade, capturando padrões periódicos (semanal, anual).
- $h(t)$: A componente de feriados, ajustando a previsão para eventos específicos.
- ϵ_t : O termo de erro, representando variações não modeladas.

Exemplo de Implementação do Prophet

A seguir, um trecho do código de `analise_prophet.py` que ilustra a inicialização e treinamento do modelo, bem como o cálculo do MAPE:

```
df = pd.read_csv(caminho_arquivo, sep=None, engine='python', encoding='u

# Prophet EXIGE que as colunas se chamem 'ds' (Data) e 'y' (Valor/Volume)
# Mapeamento flexível das colunas do seu sistema
col_map = {col: 'ds' for col in df.columns if 'data' in str(col).lower()}
col_map.update({col: 'y' for col in df.columns if 'volume' in str(col).l

df = df.rename(columns=col_map)

# Tratamento do formato da data e de números brasileiros ("1.500,00" ->
df['ds'] = pd.to_datetime(df['ds'], format='%d/%m/%Y', errors='coerce')
df['y'] = df['y'].apply(lambda x: str(x).replace('.', '').replace(',', ''

# Limpa linhas inválidas
df = df.dropna(subset=['ds', 'y']).sort_values(by='ds')

return df
except Exception as e:
    print(f"❌ Erro ao ler os dados reais ({e}). Utilizando simulador como f
    return simular_historico_impressoes()
```

7.2. Motor de Negócios (Cálculo do IPL)

Equação do Índice de Prioridade Logística (IPL)

O IPL é calculado através de uma ponderação de fatores normalizados, conforme a equação:

$$\text{IPL} = (\text{Volume_Norm} * 0.20) + (\text{Peso_Tipo} * 0.30) + (\text{Perf_Norm} * 0.25) + (\text{Logistica_Norm} * 0.25)$$

Onde: - **Volume_Norm** : Volume de vazão normalizado (0 a 1). - **Peso_Tipo** : Peso semântico do tipo de serviço (ex: Perícia = 1.5, SEA = 1.0). - **Perf_Norm** : Performance logística normalizada e invertida (pior performance = maior peso). - **Logistica_Norm** : Dificuldade/custo logístico normalizado (maior custo = maior peso).

Exemplo de Implementação do Cálculo do IPL

Um trecho do código de **geradordepesos.py** demonstra a normalização das métricas e o cálculo do IPL:

```
# Cidades mais distantes/custosas recebem maior peso logístico
l_min, l_max = df_final['Dificuldade_Logistica'].min(), df_final['Dificuldade_Logistica'].max()
df_final['Logistica_Norm'] = (df_final['Dificuldade_Logistica'] - l_min) / (l_max - l_min)

# Atribuição de Peso Semântico (Baseado na Ontologia)
# Perícias possuem peso 1.5 (Urgência Social) e SEA peso 1.0
df_final['Peso_Tipo'] = df_final['Tipo'].apply(lambda x: 1.5 if 'PERICIA' in str(x) else 1.0)

# Cálculo do IPL (Índice de Prioridade Logística)
# Nova Fórmula com Performance e Logística
# Pesos distribuídos: Volume (20%), Tipo (30%), Performance (25%), Logística (25%)
df_final['IPL'] = (
    (df_final['Volume_Norm'] * 0.20) +
    (df_final['Peso_Tipo'] * 0.30) +
    (df_final['Perf_Norm'] * 0.25) +
    (df_final['Logistica_Norm'] * 0.25)
)

def gerar_grafico_ipl(df):
    """Gera e salva um gráfico de barras com as maiores prioridades de IPL."""
    print("Gerando gráfico de prioridade IPL...")
```

7.3. Simulador Financeiro (Monte Carlo)

Exemplo de Implementação da Simulação Monte Carlo

O `teststress.py` utiliza uma simulação de Monte Carlo para comparar cenários. Abaixo, o núcleo da lógica de simulação:

```
def simular_testes_monte_carlo(iteracoes=1000000, cidades_rota=14, total_cidades=20):
    print(f"🚀 Iniciando Simulação Monte Carlo com {iteracoes:,} iterações (Rota de {cidades_rota} cidades)")

    # Simulação vetorizada com NumPy (Processamento massivo em milissegundos)
    # Simulação de custo aleatório (Cenário Atual visitando TODAS as cidades)
    custo_atual = np.random.uniform(500, 800, iteracoes)

    # Otimização 1: Redução por não visitar cidades de baixa prioridade (14 de 20)
    fator_cidades = cidades_rota / total_cidades
```

8. Informações do Ambiente e Hardware

Detalhes do ambiente de execução e recomendações de hardware para o projeto.

8.1. Ambiente de Execução

- **Sistema Operacional:** `Linux 6.14.0-15-generic (#15-Ubuntu SMP PREEMPT_DYNAMIC Sun Apr 6 15:05:05 UTC 2025)`
- **Arquitetura da Máquina:** `x86_64`
- **Versão do Python:** `3.13.11`
- **Nome do Host:** `eduardonote-Inspiron-15-3530`

8.2. Recomendações de Hardware

Para garantir a performance ideal do sistema, especialmente para o treinamento do modelo Prophet e manipulação de grandes volumes de dados com Pandas, as seguintes especificações de hardware são recomendadas: - **Processador (CPU):** Múltiplos núcleos (Dual-Core 2.0GHz ou superior) — O algoritmo de *fitting* do Prophet

se beneficia em cálculos matemáticos pesados. - **Memória RAM:** 4 GB mínimo (8 GB recomendado, para suportar o carregamento em memória **RAM** de extensas planilhas de volumetria via Pandas). - **Armazenamento:** ~1 GB de espaço livre para comportar os binários das bibliotecas Python (**site-packages**) e geração dos *outputs* (CSVs e gráficos).

Apêndice: Código-Exemplo (Open Repository)

Trecho base da arquitetura do Motor de Prioridade Multicritério:

```
# Cálculo Estrito do Índice de Prioridade Logística (IPL)
df['IPL'] = (
    (df['Volume_Norm'] * 0.20) +      # Necessidade de Vazão Quantitativa
    (df['Peso_Tipo'] * 0.30) +        # Criticidade Regulatória (Perícia = 1.5)
    (df['Perf_Norm'] * 0.25) +        # Risco de Rompimento de SLA
    (df['Logistica_Norm'] * 0.25)     # Custo / Dificuldade de Deslocamento
)
```

Nota: O código-fonte do pipeline é modular e os scripts integrais (.py) estão disponíveis no repositório logístico central para auditoria.